객체지향 프로그래밍 기초

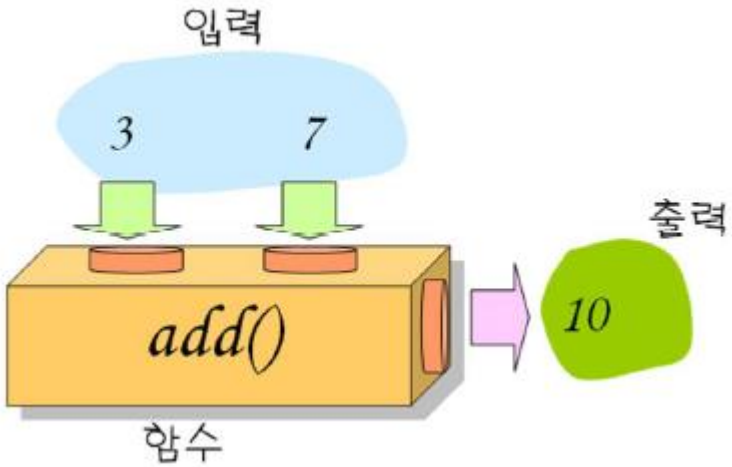
7강. 함수

“크고 복잡한 문제를
작게 나눠서 하나씩 해결해 나가자!”

함수의 정의

- 함수는 특정한 작업을 수행할 수 있도록 정의한 것이다.
- C++ 프로그램은 하나 이상의 함수로 구성된다.
- 제일 먼저 호출되는 함수는 main() 함수이다.





함수의 구조

```
#include <iostream>

int main()
{
    cout << "이것이 함수이다.\n";
    return 0;
}
```



```
#include <iostream>

void showData();

int main()
{
    showData();
    return 0;
}

void showData()
{
    cout << "이것이 함수이다.\n";
}
```

①함수의 선언(원형

③함수의 호출

②함수의 정의

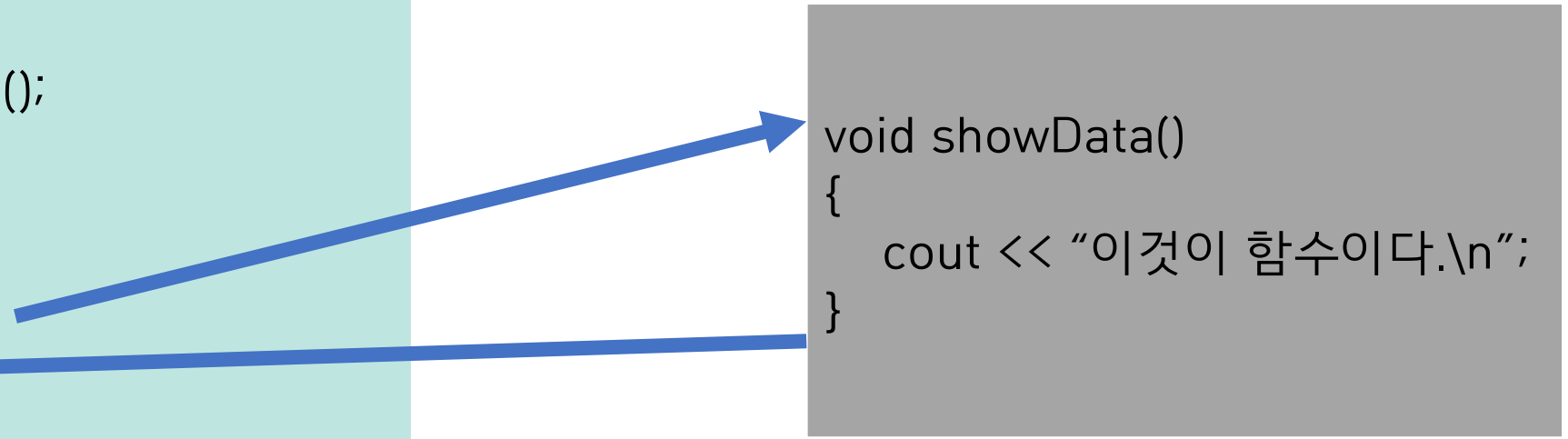
함수의 구조

```
#include <iostream>

void showData();

int main()
{
    showData();
    return 0;
}

void showData()
{
    cout << "이것이 함수이다.\n";
}
```

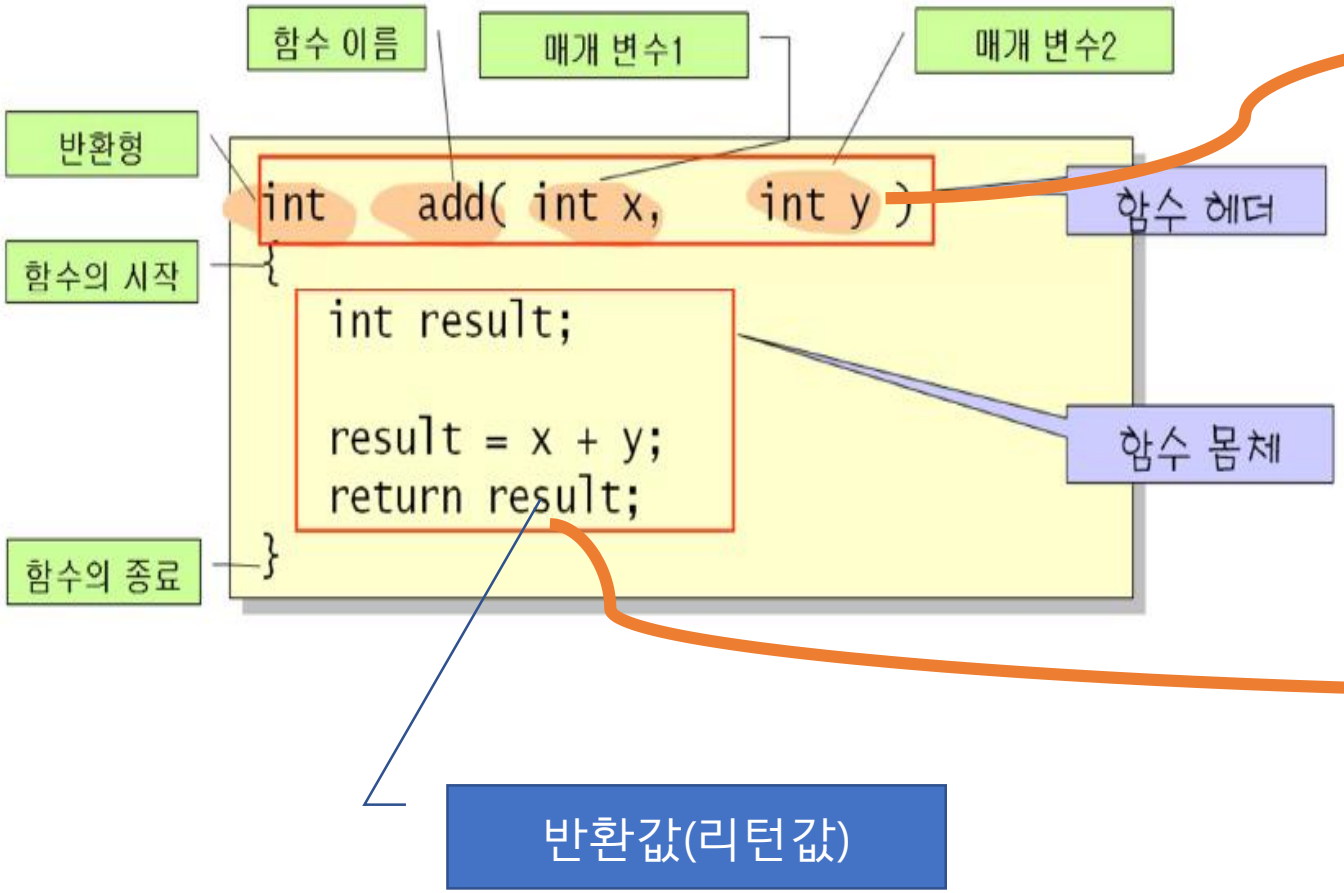


```
void showData()
{
    cout << "이것이 함수이다.\n";
}
```

함수의 종류

- C++ 표준 라이브러리 함수
  - cout, cin 등과 같이 미리 만들어진 유용한 함수들을 모아놓은 라이브러리
  - 라이브러리 함수를 사용하려면 그 함수의 정보가 저장된 헤더파일을 포함해야 한다.
- 사용자 함수
  - 사용자가 직접 만든 사용자 정의 함수

함수 정의

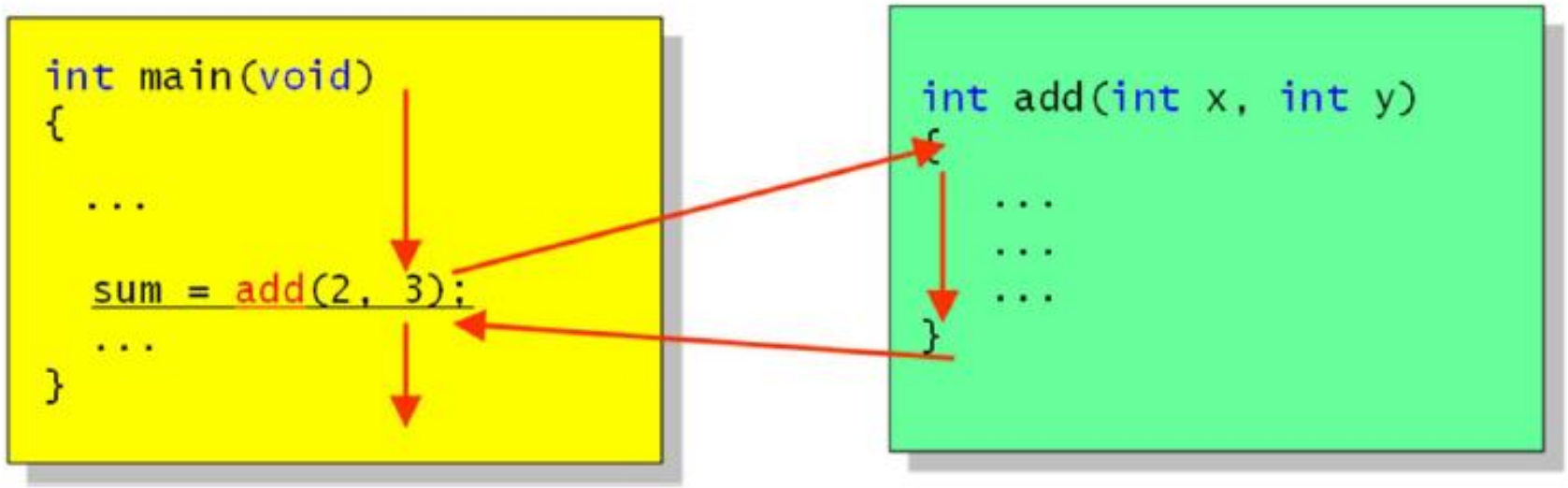




© 2012 생능출판사 All rights reserved

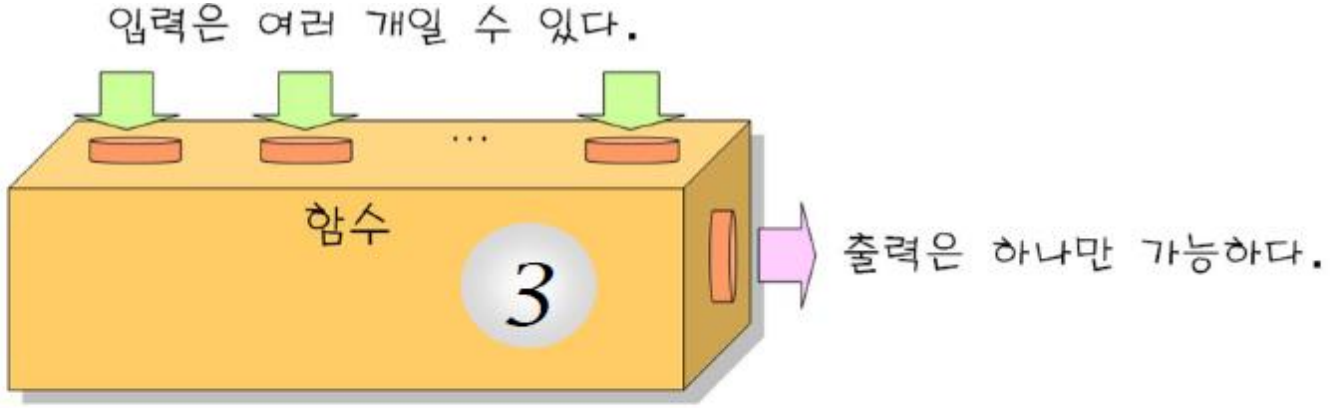
함수 호출과 반환

- 함수를 정의한 후, 프로그램에서 함수를 사용하는 것을 함수호출(function call)이라고 한다.
- 함수를 호출하면 함수 정의 위치로 이동하면서 인자값을 전달받아 매개변수에 대입한다.
- 함수몸체를 실행한 후 호출한 함수 다음 위치로 돌아온다.
- 이 때 실행한 함수는 결과값을 리턴하는 경우와 리턴하지 않는 경우로 구분된다.



반환값

- 반환값(return value)은 호출된 함수가 호출한 곳으로 작업의 결과값을 전달하는 것
- 매개변수는 여러 개가 가능하나 반환값은 하나만 가능




```
return 0;
return(0);
return x;
return x*x+2*x+1;
```

© 2012 생능출판사 All rights reserved

반환값

① void형 함수(반환값이 없는 함수)

```
void showData(int num);
int main()
{
        int n;
        cin >> n;
        showData(n);
}
```

```
void showData(int num)
{
        cout << num << endl;
}
```

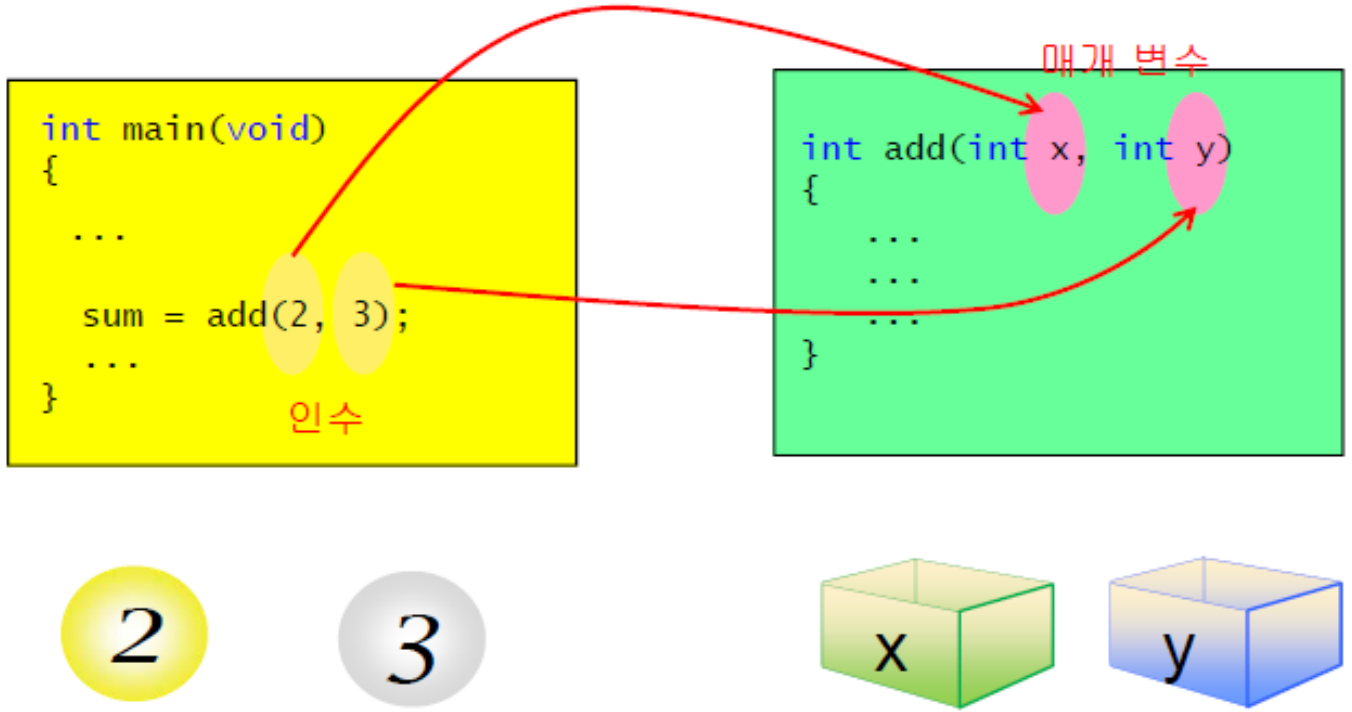
② void형이 아닌 함수 : 반환값이 있는 함수

```
int AddData(int num);
int main()
{
        int n;
        cin >> n;
        AddData(n);
}
```

```
int AddData(int num)
{
        int sum;
      sum = num + 100;
      return sum;
}
```

인수와 매개변수

- 매개변수는 외부에서 전달되는 데이터가 저장되는 변수



© 2012 생능출판사 All rights reserved

함수 선언

- 함수를 정의하기 전에 함수를 선언해야 한다.
- 함수선언은 함수이름, 매개변수, 함수가 리턴하는 값의 자료형을 기술하는 것으로 '함수 원형 ' 이라고 한다.
- 함수 선언의 방법

  ① 특정한 파일에 함수 원형을 선언한 후 메인소스파일에 #include <특정한 파일> 로 지정한다.

  ② 소스코드 내에서 함수를 호출하는 것이 함수 정의보다 앞에 위치하면 함수 원형을 선언한다.

  ③ 소스코드 내에서 함수를 호출하는 것이 함수 정의보다 뒤에 위치하면 함수 원형을 선언하지 않는다.

함수 선언 : 함수 원형

```
#include <헤더파일>
using namespace std;
```

```
void 함수명(매개변수)
```

```
int main()
{
        함수명(인수);
        return 0;
}
```

```
void 함수명(매개변수)
{
        몸체;
}
```

```
#include <헤더파일>
using namespace std;
```

```
자료형 함수명(매개변수)
{
        몸체;
        return 반환식;
}
```

```
int main()
{
        함수명(인수);
        return 0;
}
```

함수 선언의 예

```
int Increment(int n)
{
    n++;
    return n;
}

int main()
{
    int num;
    num = Increment(num);
    return 0;
}
```

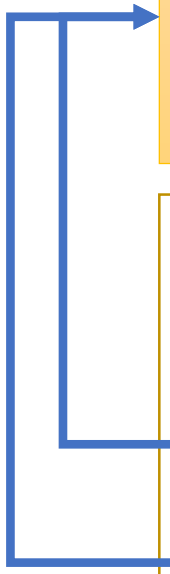
```
int Increment(int n);

int main()
{
    int num;
    num = Increment(num);
    return 0;
}

int Increment(int n)
{
    n++;
    return n;
}
```

함수의 장점

```
int main()
{
    int i;
    for (i=0; i<10; i++)
            printf("*");
    . . .
    for (i=0; i<10; i++)
            printf("*");
    . . .
    for (i=0; i<10; i++)
            printf("*");
    . . .
}
```



```
void print_star()
{
    for (i=0; i<10; i++)
            printf("*");
}
```

```
int main()
{
    int i;
    print_star();
    . . .
    print_star();
    . . .
    print_star();
    . . .
}
```

함수의 장점

- 크고 복잡한 문제를 작고 쉬운 문제들로 나누어 해결하기 쉽도록 한다.

  =>구조적인 프로그래밍

- 같은 코드를 반복적으로 기술해야 하는 중복을 최소화하여 코드양을 줄인다.

  => 코드의 수정, 유지보수가 쉬워진다.

- 캡슐화가 가능하다.

  => 함수가 어떻게 구현되었는지 몰라도 사용법만 알면 사용가능하다.

[예제]함수 구현

두 개의 정수를 입력받은 후, 두 수를 인자로 전달받아 큰 수를 반환하는 함수를 정의하고 함수로부터 반환받은 수를 출력하는 프로그램

```
int main()
{
        int x1, x2, max;
        cout << " 두 개의 정수를 입력하시오.: ";
        cin >> x1 >> x2;
        if (x1 > x2)
                max= x1;
        else
                max = x2;
        cout << "더 큰 수는 : " << max << "입니다.\n";

        return 0;
}
```

```
int NumCompare(int num1, int num2);

int main()
{
	int x1, x2;
	cout << " 두 개의 정수를 입력하시오.: ";
	cin >> x1 >> x2;
	cout << "더 큰 수는 : " << NumCompare(x1, x2) << "입니다.\n";
	return 0;
}
int NumCompare(int num1, int num2)
{
	int result;
	if (num1 > num2)
		result = num1;
	else
		result = num2;
	return result;
}
```

[예제1]

한 개의 정수를 입력받은 후, 짝수와 홀수를 판별하는 프로그램을 작성하되, even() 함수를 이용하여 짝수/홀수를 판별하라.

even()함수 :

- 짝수, 홀수를 판별하여 true 및 false값을 반환하는 함수
- 매개변수 : 정수값
- 리턴형 : bool 타입
- 리턴값 : true 또는false;

[예제2]

입력받은 문자가 대문자면 소문자로, 소문자면 대문자로 변환하는 프로그램을 작성하라.
대문자는 소문자로, 소문자는 대문자로 변환하는 함수를 사용하라.

[예제3]

두 정수를 입력받고, 두 정수에 대한 평균을 구하는 프로그램을 작성하라.
평균을 구하는 함수를 사용하라.

[예제4]

아래는 정수를 입력받아, 1부터 정수까지 곱하는 프로그램이다. 빨간 박스 부분을 함수로 구현하라.

```
int main()
{
        int x;
        int result = 1;

        cin >> x;
        for (int i=1; i<=n; i++)
                result *= i;

        cout << result << endl;

        return 0;
}
```

[예제]

세 개의 정수를 입력받은 후, 세 정수를 인자로 전달받아 그 중 가장 큰 수를 반환하는 함수와 가장 작은 수를 반환하는 함수를 정의하시오. 그리고 이 함수를 호출하여 각각의 값을 출력하는 프로그램을 작성하시오.

[과제]

아래의 결과가 나오도록 프로그래밍하시오.

메뉴설명

① 정수 n과 k를 입력받아 n의 k승을 계산하여 출력(함수명:power())

② 자연수 n을 입력받아 2부터 그 수 사이에 있는 소수를 모두 찾아 출력 (PrimeNumber())

오버로드(overload)

- 오버로드는 여러 함수들이 동일한 이름을 사용할 수 있는 기능을 말한다.

(함수의 이름은 똑같으면서 매개변수의 종류(데이터타입, 매개변수의 개수)만 다른 경우)

```
int add(int a, int b);
float add(float a, float b);
double add(double a, double b);
```

오버로드(overload)

```
int add(int a, int b)
{
        int sum;
        sum = a + b;
        return sum;
}
float add(float a, float b) {
        return a + b;
}
double add(double a, double b) {
        return a + b;
}
```

```
int main()
{
        int r1 = add(1, 2);
        float r2 = add(1.0f, 2.0f);
        double r3 = add(1.0, 2.0);
        return sum;
}
```

오버로드의 규칙

① 어떤 함수를 호출할지 결정하는 규칙

인자의 타입을 확인해서 그에 가장 잘 어울리는 시그니처를 가진 함수를 호출

```
int VariableReturnType(char c, int i);
double VariableReturnType(char c, double i);

VariableReturnType('A', 100);
VariableReturnType('A', 100.12);
```

오버로드의 규칙

② 반환값만 틀린 경우는 오버로드할 수 없다.

```
int VariableReturnType(char c, int i);
double VariableReturnType(char c, int i);

VariableReturnType('A', 100);
```

[예제]

정수의 제곱과 실수의 제곱을 구하는 함수 square()를 작성하되, 오버로딩하시오.

디폴트 매개변수

- 디폴트 매개변수는 값을 지정해 주지 않았을 때 기본적으로 선택하는 값을 의미한다.
- 만약 함수의 매개변수에 대응하는 인자가 함수의 호출 시에 생략되면 매개변수에 기본 값이 복사된다.
- 디폴트 매개변수는 맨 뒤에서부터 앞쪽으로 순서대로 채워서만 정의할 수 있다.

디폴트 매개변수

```
int GetNumber(int p1=10)
{
        . . .
        return i;
}

int main()
{
        GetNumber();
}
```

디폴트 매개변수

- 디폴트 매개변수는 맨 뒤에서부터 앞쪽으로 순서대로 채워서만 정의할 수 있다.

```
int add(int p1, int p2, int p3=30);
int add(int p1, int p2=20, int p3=30);
int add(int p1=10, int p2=20, int p3=30);

int add(int p1, int p2=20, int p3);
int add(int p1, int p2, int p3=30);
int add(int p1=10, int p2, int p3=30);
```

```
void DefaultArg(int p1, int p2=2, int p3=30);

int main()
{
        DefaultArg(100);
        return 0;
}
```

```
void DefaultArg(int p1=1, int p2=2, int p3=30);
int main()
{
        DefaultArg(100,200);
        return 0;
}
```

inline 함수

- 인라인함수는 실행 속도를 빠르게 하기 위한 것이다.
- 일반적인 함수의 호출은 프로그램을 다른 주소(함수의 주소)로 점프시켰다가, 함수의 처리가 종결되면 다시 원래의 자리로 돌아온다.
- 인라인함수는 컴파일된 함수 코드가 프로그램의 다른 코드 안에 직접 삽입된다. 즉, 인라인 코드를 이용하면 함수를 그 자리에서 처리한다.

➡ 일반 함수보다 빠르게 수행되지만 메모리 사용 측면에서는 불리하다.

➡ 함수 코드를 수행하는 데 걸리는 시간이 매우 짧을 경우에만 인라인 함수를 사용하는 것이 효율적이다.

inline 함수

```
#include <iostream>
using namespace std;

inline int calc(int x) { return 2*x; }

int main()
{
        int x=5;
        int y=7;
        cout << calc(x) << endl;
        cout << calc(y) << endl;
        return 0;
}
```

[게임을 위한 라이브러리]난수 발생 함수

- rand() : 임의의 정수값을 반환하는 함수
  보통 나머지(%) 연산을 이용해 원하는 범위의 수로 변환한다.

```
int isFace = rand() % 2;            // 0~1의 숫자 무작위 발생
int diceNum = rand() % 6 +1    // 1~6의 숫자 무작위 발생
```

- 난수발생이 실제로는 이미 정해진 난수표(random number table)에 의해 이루어지고, 항상 같은 위치에서부터 값을 출력하므로 프로그램을 실행할 때마다 동일한 숫자들이 순서대로 발생된다.
- 프로그램 실행시마다 다른 순서의 난수를 발생시키려면 표에서 시작(seed)위치를 바꾸어야 한다. -> srand()

난수 발생 함수

- srand() : 무작위 시작점 설정
  하나의 매개변수가 필요한데 보통 현재 시각 정보를 사용한다.(time())

  <time.h>, <stdlib.h>
  무작위 시작점 설정 : srand((unsigned)time(NULL))
  무작위값 발생 함수 : rand()
  무작위값의 범위 조정 : rand() % MAX

동전을 던져서 앞면/뒷면이 나오는 결과

```
#include <iostream>
#include <stdlib.h>
#include <time.h>
using namespace std;

inline int randCoin() { return rand() % 2; }

int main()
{
	srand((unsigned)time(NULL));
	cout << randCoin() << endl;
	return 0;
}
```

[예제]게임

Play Up & Down 게임을 아래와 같이 프로그래밍하시오.

(1) 컴퓨터가 0~10 사이의 숫자를 임의로 설정하고, 사용자에게 보여주지는 않는다.

(2) 게이머는 숫자를 예측하고 프로그램은 정답과 비교하여 “더 큰 숫자입니다.”나 “더 작은 숫자입니다.”, “정답입니다.”를 출력한다.

(3) 세 번 내에 맞추거나, 세 번 내에 맞추지 못하면 게이머의 게임 점수를 출력하고 게임은 종료된다.(프로그램의 종료가 아님)

(4) 게임 점수는 몇 번 만에 맞추는지에 따라 결정된다.(한번에 맞추면 100점, 두 번에 맞추면 90점, 세 번에 맞추면 80점, 세 번 내에 맞추지 못하면 0점이다.)

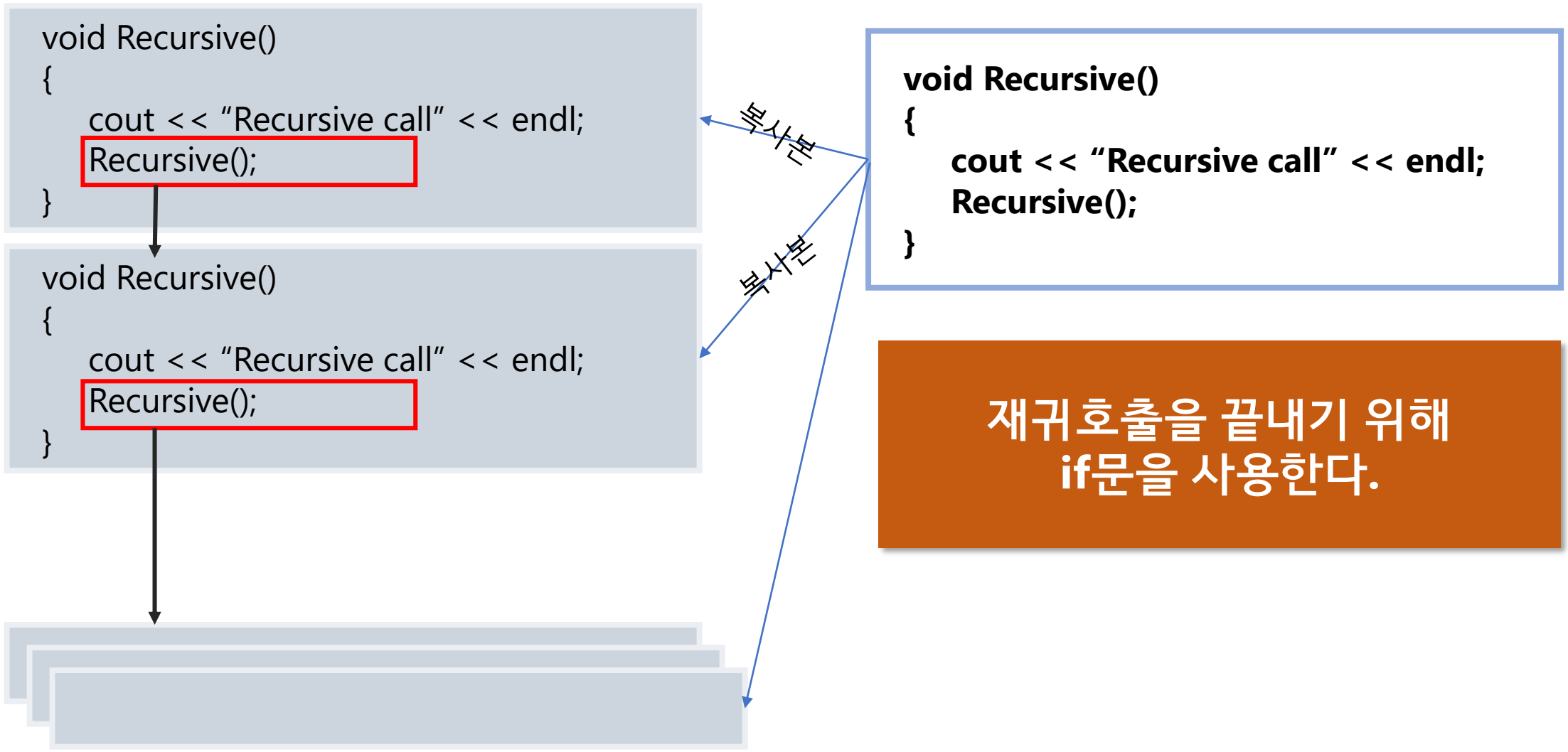
(5) 게이머에게 게임을 다시 할 것인지 묻는다. (Y/N)

재귀함수

함수 내에서 자기 자신을 다시 호출하는 함수

```
void Recursive()
{
    cout << “Recursive call” << endl;
    Recursive();
}
```

재귀함수


void Recursive()
{
cout << “Recursive call” << endl;
Recursive();
}
void Recursive()
{
cout << “Recursive call” << endl;
Recursive();
}
복사본
복사본
void Recursive()
{
cout << “Recursive call” << endl;
Recursive();
}
재귀호출을 끝내기 위해
if문을 사용한다.

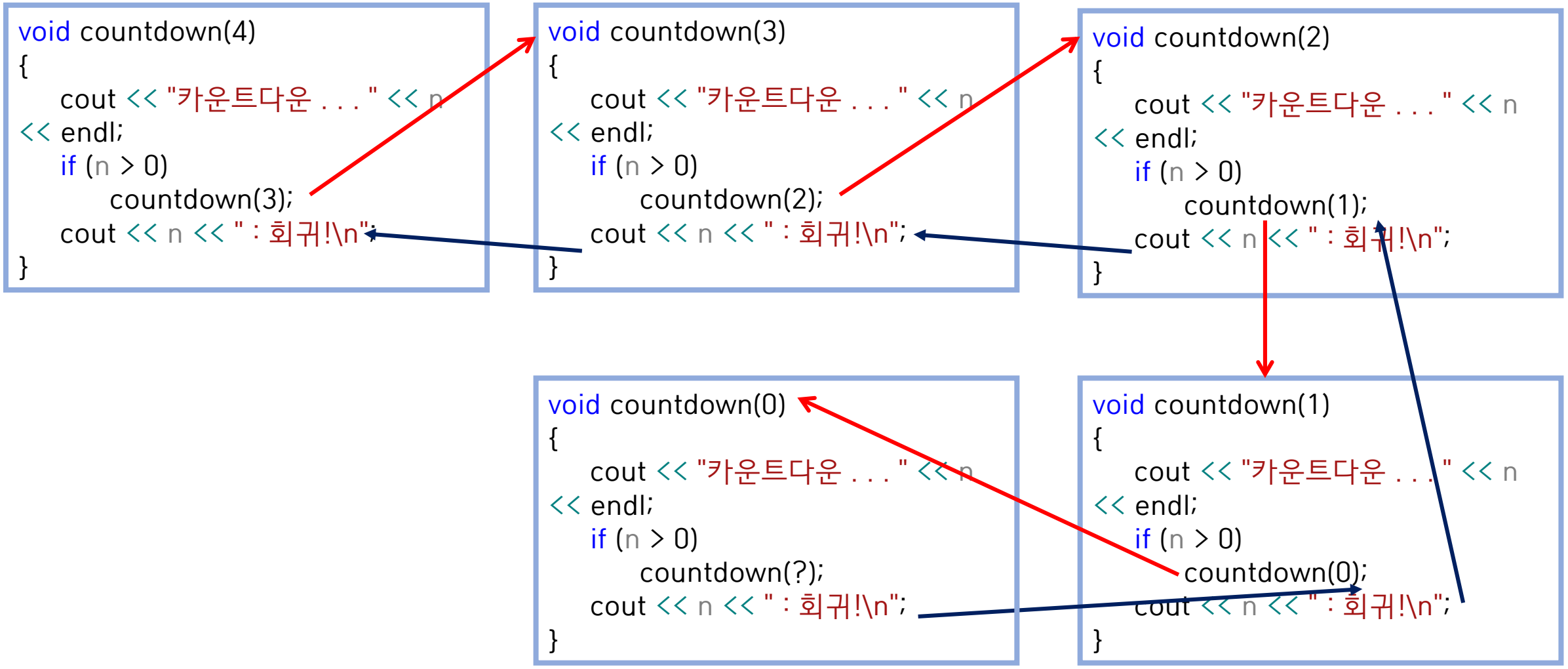
재귀함수의 호출

```
#include <iostream>
using namespace std;

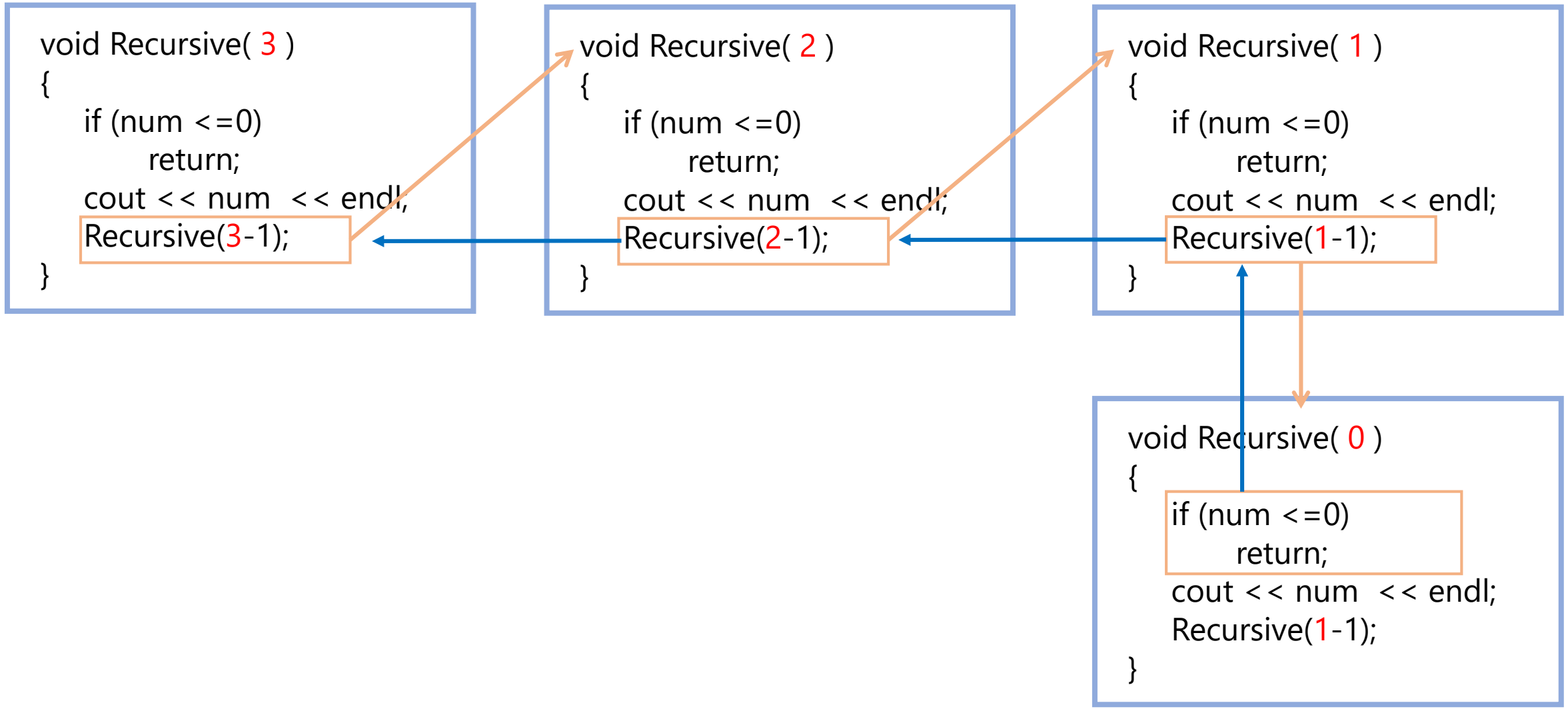
void countdown(int n);
int main()
{
    countdown(4);
    return 0;
}
void countdown(int n)
{
    cout << "카운트다운 . . . " << n << endl;
    if (n > 0)
        countdown(n - 1);
    cout << n << " : 회귀!\n";
}
```

```
카운트다운 . . . 4
카운트다운 . . . 3
카운트다운 . . . 2
카운트다운 . . . 1
카운트다운 . . . 0
0 : 회귀!
1 : 회귀!
2 : 회귀!
3 : 회귀!
4 : 회귀!
```

재귀함수의 호출


void countdown(4)
{
cout << "카운트다운 . . . " << n
<< endl;
if (n > 0)
countdown(3);
cout << n << " : 회귀!\n";
}
void countdown(3)
{
cout << "카운트다운 . . . " << n
<< endl;
if (n > 0)
countdown(2);
cout << n << " : 회귀!\n";
}
void countdown(2)
{
cout << "카운트다운 . . . " << n
<< endl;
if (n > 0)
countdown(1);
cout << n << " : 회귀!\n";
}
void countdown(0)
{
cout << "카운트다운 . . . " << n
<< endl;
if (n > 0)
countdown(?);
cout << n << " : 회귀!\n";
}
void countdown(1)
{
cout << "카운트다운 . . . " << n
<< endl;
if (n > 0)
countdown(0);
cout << n << " : 회귀!\n";
}

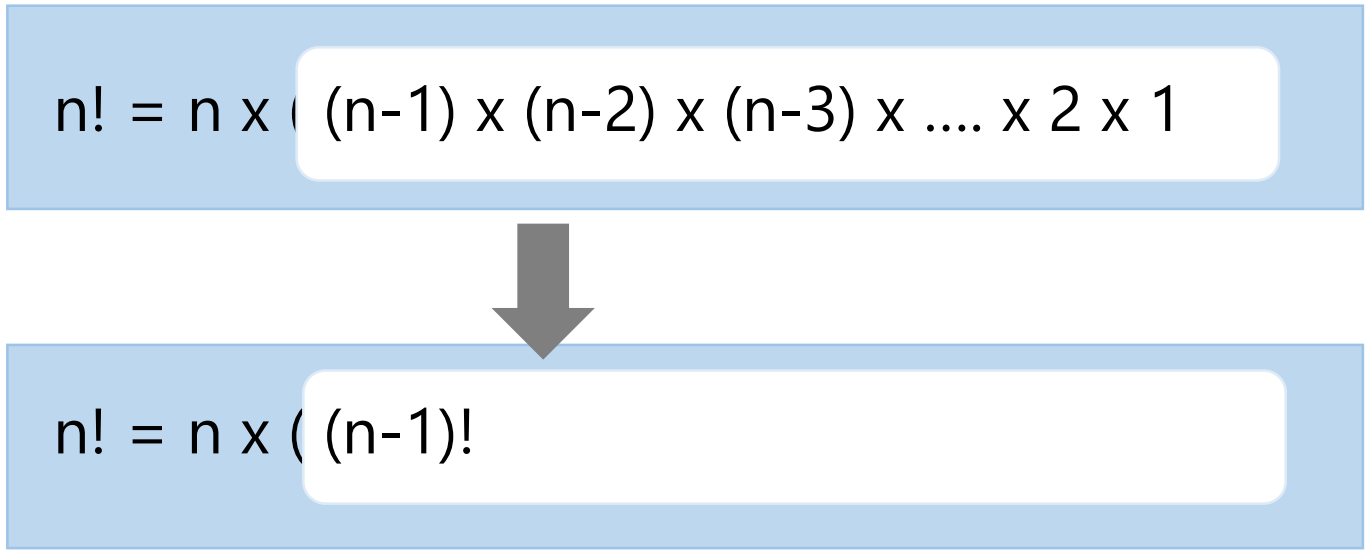
재귀함수의 호출과정


void Recursive(3)
{
if (num <=0)
return;
cout << num << endl;
Recursive(3-1);
}
void Recursive(2)
{
if (num <=0)
return;
cout << num << endl;
Recursive(2-1);
}
void Recursive(1)
{
if (num <=0)
return;
cout << num << endl;
Recursive(1-1);
}
void Recursive(0)
{
if (num <=0)
return;
cout << num << endl;
Recursive(1-1);
}

재귀함수의 디자인

재귀함수는 자료구조나 알고리즘의 어려운 문제를 단순화하는데 사용됨

$$n! = n \times (n-1) \times (n-2) \times (n-3) \times \ldots \times 2 \times 1$$



$$f(n) = \begin{cases} n \times f(n-1) & \text{if } n >= 1 \\ 1 & \text{if } n=0 \end{cases}$$

재귀함수의 디자인

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

int main()
{
    cin >> i;
    cout << i << " ! = " << factorial(i) << endl;
    return 0;
}
```

[예제4]

(1) 1부터 n까지의 누적합을 구하는 재귀함수 프로그램을 작성하라.

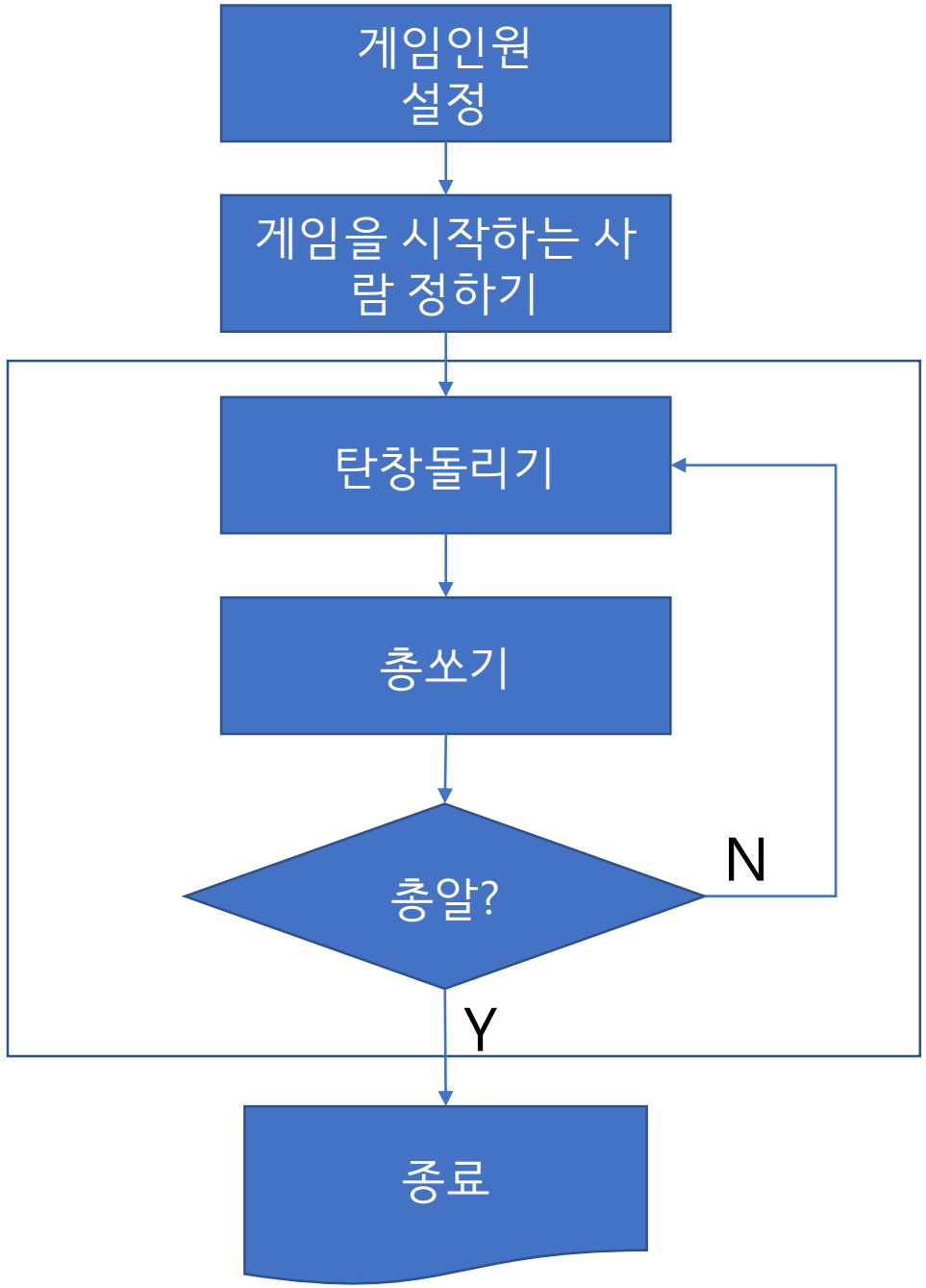
(2) 2의 n승을 구하는 재귀함수 프로그램을 작성하라.

(2의 n승 : 2x2x2x2x……)

[과제]러시안 룰렛

러시안 룰렛 게임의 규칙은 아래와 같다.

- 게임인원은 5명이다.
- 6연발 권총으로 가정하고, 총알의 개수는 1개이다.
- 1번 이용자부터 게임을 시작한다.
- 방아쇠를 당기기 전, 탄창을 돌려야 한다.
- 한 사람이 방아쇠를 당겨 총에 맞으면 게임이 끝난다. 그렇지 않으면 다음 사람이 탄창을 돌린 후 방아쇠를 당긴다.


게임인원
설정
게임을 시작하는 사
람 정하기
탄창돌리기
총쏘기
총알?
N
Y
종료

```
#include "Myfunction.h"

int main()
{
    int member, nBullets;
    srand((unsigned)time(NULL));

    cout << "게임인원 : "; cin >> member;
    cout << "총알을 넣으세요 : "; cin >> nBullets;
    getchar();
    int bang = playRussianRoulette(member, nBullets);
    cout << bang << "번째 참가자가 총에 맞았습니다." << endl;

    return 0;
}
```

```
#include "Myfunction.h"

int playRussianRoulette(int nTurns, int nBullets)
{
    int start = rand() % nTurns;
    cout << "총을 돌렸습니다." << start + 1 << "번부터 시작합니다." << endl << endl;
    while (true)
    {
        int pos = rand() % 6;
        cout << "탄창을 돌렸습니다."<<endl;
        cout << start+1 << "번째 참가자가 엔터키를 누르면 총이 발사됩니다.₩n";
        getchar();
        if (pos == nBullets) break;
        start = (start + 1) % nTurns;
    }
    return start + 1;
}
```

```
#pragma once
#include <iostream>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
using namespace std;
extern int playRussianRoulette(int nTurns, int nBullets);
```

[예제3]스피드 구구단

10개의 구구단 문제가 주어지면 최대한 빨리 답을 맞추는 게임이다.

- 곱셈은 2~9사이의 수로 이루어진다.
- 10개의 구구단 문제가 출제되고 문제를 푸는데 걸린 전체 시간을 계산한다.
- 이 때 한 문제라도 답이 틀린 경우 게임이 종료되고 10개의 문제를 다 맞춘 경우 전체 소요시간을 출력한다.

시간측정 함수

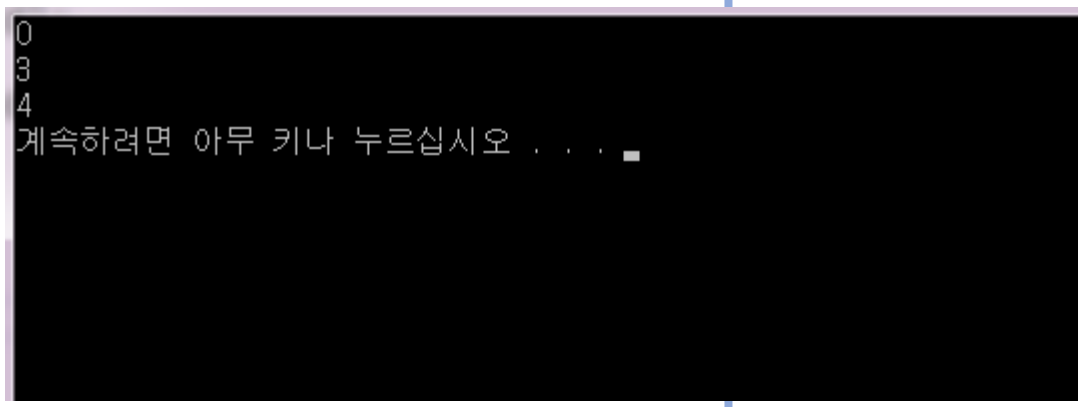
- clock() : 호출되었을 때의 시스템 시각을 clock단위로 반환한다.
- 실행시간을 측정하기 위해서는 두 시점에서의 시각을 구하고 그 차이를 CLOCKS_PER_SEC로 나누면 초(second)단위의 시간이 된다.

```
clock_t t1 = clock();
~~
clock_t t2 = clock();
double duration = (double)(t2-t1) / CLOCKS_PER_SEC;
```

[복습]다음 프로그램의 실행결과는?

```
#include <iostream>
using namespace std;
void Add(int val);
int num;

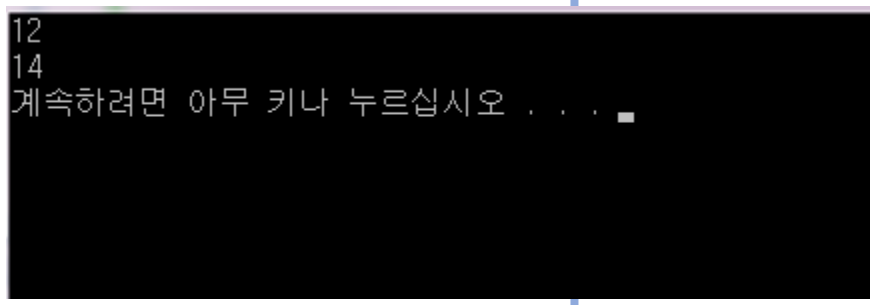
int main()
{
	cout << num << endl;
	Add(3);
	cout << num << endl;
	num++;
	cout << num << endl;
	return 0;
}
void Add(int val)
{
	num += val;
}
```



[복습]다음 프로그램의 실행 결과는?

```
#include <iostream>
using namespace std;
int Add(int val);
int num=1;

int main()
{
	int num = 5;
	cout << Add(3) <<endl;
	cout << num + 9 <<endl;
	return 0;
}
int Add(int val)
{
	int num = 9;
	num += val;
	return num;
}
```



[복습]다음 프로그램의 실행 결과는?

```
#include <iostream>
using namespace std;
void SimpleFunc()
{
	static int num1 = 0;
	int num2 = 0;
	num1++; num2++;
	cout << "static: "<< num1<< "local: "<< num2 << endl;
}

int main()
{
	int i;
	for (i = 0; i < 3; i++)
	SimpleFunc();
	return 0;
}
```

